

PT1 Stage 2: Conceptual and Logical Database Design

Project: Lights Out Trivia

Team: 058

Team Members: Krishna Balaji, Chirag Seth, Dhruv Doshi, Rohan Jain

UML Diagram

Diagram: CS411-JOINTheDarkSide-PT1-UML

Added image and the source code for the [draw.io](#) diagram in Github.

Scope and General Assumptions

Our Formula 1 data comes from the Kaggle Formula 1 World Championship dataset, based on Ergast data, and FastF1. Kaggle provides the main race, driver, constructor, standings, lap, qualifying, and pit stop data. FastF1 adds detailed lap and telemetry information.

The database covers Formula 1 from 1979 through 2024. We start in 1979 because earlier races could have the same driver appear more than once in one race due to shared drives. From 1979 onward, each (race, driver) pair is unique, which allows us to use that combination to identify a RaceResult.

FastF1 currently has more limited coverage than the Kaggle data, so FastF1-specific attributes can be empty for races where that data is unavailable. Telemetry is stored only for selected laps, mainly each driver's fastest lap, because storing hundreds of samples for every lap would create a large amount of unnecessary data.

Admins are not modeled separately. An admin is a User with isAdmin = true.

Entity Assumptions

Entity	Primary / Composite Key	What it represents	Assumptions and design decisions
Season	year	One Formula 1 championship year	Modeled separately because races and championship standings belong to a specific season.
Circuit	circuitId	Physical Formula 1 track	circuitRef is unique. Circuit is separate from Race because one circuit can host many races, avoiding repeated circuit information. Historical layout changes are not modeled separately.
Race	raceId	Main Grand Prix race of an event weekend	(year, round) is also unique. Race is separate because results, qualifying information, and other race-specific data depend on a particular race. Sprint races, practice sessions, and session schedules are outside the current scope.
Driver	driverId	Individual Formula 1 driver	driverRef is unique. Driver code and permanent number are not keys because they can be missing or reused. A driver's actual number in a race is stored as carNumber on RaceResult.
Constructor	constructorId	Formula 1 team	constructorRef is unique. Constructor is separate from Driver because a driver can change teams. It is associated with RaceResult instead. Different team brandings

			are treated as separate constructors.
RaceResult	(raceId, driverId)	One driver's result in one race	Weak entity. It stores result-specific information such as grid position, finishing position, points, laps completed, and status. The race and driver together uniquely identify a result from 1979 onward. status remains an attribute because it has no additional data of its own.
Lap	(raceId, driverId, lapNumber)	One driver's lap in a race	Weak entity belonging to RaceResult. It is separate because one result can have many laps. tyreCompound and trackStatus remain attributes because they do not require their own entities.
PitStop	(raceId, driverId, lapNumber)	Pit stop during a lap	Weak entity owned by Lap. A driver can make at most one pit stop during a particular lap. It is separate from Lap because most laps do not contain a pit stop.
TelemetrySample	(raceId, driverId, lapNumber, offsetMs)	One telemetry reading within a lap	Weak entity. A lap can contain hundreds of telemetry readings, so telemetry cannot reasonably be stored as Lap attributes. offsetMs identifies the reading within the lap.
User	userId	Application account	username and email are unique. There is only one account entity; admins are represented using isAdmin. Leaderboards

			are calculated from quiz attempts rather than stored as entities.
Quiz	quizId	Collection of trivia questions	Can be SYSTEM, CUSTOM, or DAILY, with PRIVATE or PUBLIC visibility. Driver, constructor, category, and era can be used as filters. Era is represented by eraStartYear and eraEndYear. A daily challenge is represented using quizType = DAILY rather than a separate entity.
Question	questionId	One trivia question	Generated questions may have a unique sourceKey. Questions can have TEXT or CHART format and can be reused across quizzes, favorites, and attempts. category, difficulty, and visibility remain attributes because they are fixed descriptive values.
AnswerOption	(questionId, label)	One possible answer to a Question	Weak entity. Questions can have different numbers of choices, and every option has its own text and correctness value. Exactly one option per Question is correct.
QuizAttempt	attemptId	One instance of a user taking a quiz	Separate entity because the same user can attempt the same quiz multiple times. startedAt and completedAt belong to the individual attempt. Score is calculated from its answers.
AttemptAnswer	(attemptId, questionId)	User's answer to one question during an attempt	Weak entity. Stores timeTakenMs and the selected AnswerOption.

			The selection may be empty if the question times out. Correctness is derived from AnswerOption.isCorrect.
--	--	--	---

Relationship Assumptions and Cardinalities

Relationship	Cardinality	Assumption / Meaning
Season includes Race	Season 1..1, Race 1..*	Each Race belongs to exactly one Season, while a Season contains multiple races.
Circuit hosts Race	Circuit 1..1, Race 1..*	Each Race is held at exactly one Circuit, while a Circuit can host multiple races.
Driver earns RaceResult	Driver 1..1, RaceResult 1..*	Each RaceResult belongs to exactly one Driver, while a Driver can have many results. This is an identifying relationship for RaceResult.
Race produces RaceResult	Race 1..1, RaceResult 1..*	Each RaceResult belongs to exactly one Race, while a Race produces multiple results. Race and Driver together identify RaceResult.
Constructor fields RaceResult	Constructor 1..1, RaceResult 1..*	Each RaceResult is associated with exactly one Constructor, while a Constructor can appear in many race results.
Driver qualified for Race	Driver 0..*, Race 0..*	Many-to-many relationship. A Driver can qualify for many races, and a Race can contain many qualifying drivers. It stores position, q1TimeMs, q2TimeMs, and q3TimeMs. The lower bound is 0 because older qualifying data may be unavailable.
Driver ranked in Season	Driver 1..*, Season 0..*	A Driver can appear in multiple seasons, while a Season ranks multiple drivers. The relationship stores final position and points.

Constructor ranked in Season	Constructor 1..*, Season 0..*	A Constructor can appear in multiple seasons, while a Season ranks multiple constructors. The relationship stores final championship position and points.
RaceResult completes Lap	RaceResult 1..1, Lap 0..*	Each Lap belongs to exactly one RaceResult, while a RaceResult can have many laps or no stored lap data. This identifies Lap through its RaceResult.
Lap has stop PitStop	Lap 1..1, PitStop 0..1	Each PitStop belongs to exactly one Lap, while a Lap can contain at most one PitStop. This is an identifying one-to-one relationship.
Lap logs TelemetrySample	Lap 1..1, TelemetrySample 0..*	Each TelemetrySample belongs to exactly one Lap, while a Lap can contain many samples or none. This is an identifying relationship.
Question visualizes Lap	Lap 0..1, Question 0..*	A Question can optionally visualize one Lap. The same Lap can be used by multiple questions.
Quiz focuses on Driver	Driver 0..1, Quiz 0..*	A Quiz can optionally focus on one Driver, while a Driver can be the focus of multiple quizzes.
Quiz focuses on Constructor	Constructor 0..1, Quiz 0..*	A Quiz can optionally focus on one Constructor, while a Constructor can be the focus of multiple quizzes.
Quiz contains Question	Quiz 0..*, Question 1..*	Many-to-many relationship. A Quiz contains one or more Questions, while a Question can appear in multiple quizzes or none. position belongs to the relationship because the same Question can appear at different positions in different quizzes.
Question offers AnswerOption	Question 1..1, AnswerOption 2..*	Each AnswerOption belongs to exactly one Question, while every Question has at least two options. This is an identifying relationship.
User owns Quiz	User 0..1, Quiz 0..*	A User can own multiple quizzes. A Quiz has at most one owner because system-generated

		quizzes do not have an owner.
User authors Question	User 0..1, Question 0..*	A User can author multiple Questions. A Question has at most one author because generated Questions have no author.
User favorites Question	User 0..*, Question 0..*	Many-to-many relationship. A User can favorite many Questions and a Question can be favorited by many Users. The relationship stores savedAt, and a User can save the same Question only once.
User takes QuizAttempt	User 1..1, QuizAttempt 0..*	Every QuizAttempt belongs to exactly one User, while a User can have many attempts.
Quiz attempted in QuizAttempt	Quiz 1..1, QuizAttempt 0..*	Every QuizAttempt is for exactly one Quiz, while the same Quiz can be attempted many times.
QuizAttempt records AttemptAnswer	QuizAttempt 1..1, AttemptAnswer 0..*	Each AttemptAnswer belongs to exactly one QuizAttempt, while an attempt can contain multiple answers. This is an identifying relationship.
Question answered in AttemptAnswer	Question 1..1, AttemptAnswer 0..*	Each AttemptAnswer refers to exactly one Question. Together with QuizAttempt, this ensures at most one AttemptAnswer exists for a Question in an attempt.
AttemptAnswer selected AnswerOption	AnswerOption 0..1, AttemptAnswer 0..*	An AttemptAnswer can select at most one AnswerOption. No option is selected if the question times out, and the selected option must belong to the same Question.

Database Normalization

The final relational schema is normalized to **BCNF**

First, Second, and Third Normal Form

All attributes are atomic, so the relations satisfy **1NF**. Values such as q1TimeMs, q2TimeMs, and q3TimeMs are fixed separate attributes rather than repeating groups.

Relations with composite keys satisfy **2NF** because their non-key attributes depend on the complete key rather than only part of it.

The schema satisfies **3NF** because the non-key attributes do not introduce transitive dependencies on other non-key attributes.

The remaining check is therefore BCNF: every non-trivial functional dependency must have a superkey as its determinant.

BCNF Decomposition

In the conceptual UML, Circuit contains both location and country.

Under our data assumption, each value of location belongs to one country. This creates the dependency:

- location -> country

Since location is not a key of Circuit, keeping both attributes in Circuit would violate BCNF.

We therefore decompose this information into:

- Location(location, country)
- Circuit(..., location, ...)

In Location, location is the primary key, so location -> country has a key as its determinant.

Location is introduced during normalization and therefore does not appear as a separate entity in the conceptual UML diagram.

BCNF Justification by Relation

Relation	Candidate Key(s)	Functional Dependency	Why It Is in BCNF
Season	year	year -> wikiUrl	The only determinant is year, which is the primary key.
Location	location	location -> country	location is the primary key, so the determinant is a key.
Circuit	circuitId; circuitRef	circuitId -> all; circuitRef -> all	After moving country to Location, location no longer determines another Circuit attribute. Name, coordinates, and altitude are not assumed to uniquely identify a circuit. The remaining determinants are candidate keys.
Driver	driverId; driverRef	driverId -> all; driverRef -> all	Driver code and permanent number are not assumed to be unique across all drivers. Both determinants are candidate keys.
Constructor	constructorId; constructorRef	constructorId -> all; constructorRef -> all	The determinants are candidate keys.
Race	raceId; (year, round)	raceId -> all; (year, round) -> all	Each Race has a unique raceId, and each round number occurs only once within a particular season. raceDate, circuitId, and name are not assumed to uniquely identify a race.
RaceResult	(raceId, driverId)	(raceId, driverId) -> all	Each driver has at most one RaceResult in a particular race under the 1979-2024 scope. Neither raceId nor driverId alone determines a result. carNumber and positionOrder are result

			attributes, not additional schema-level keys.
Qualifying	(raceId, driverId)	(raceId, driverId) -> position, q1TimeMs, q2TimeMs, q3TimeMs	Each row stores one driver's qualifying result for one race. Neither raceId nor driverId alone determines the qualifying values.
DriverStanding	(year, driverId)	(year, driverId) -> position, points	Each row stores one driver's final standing for one season. Neither year nor driverId alone determines the standing values.
ConstructorStanding	(year, constructorId)	(year, constructorId) -> position, points	Each row stores one constructor's final standing for one season. The complete key determines the remaining values.
Lap	(raceId, driverId, lapNumber)	(raceId, driverId, lapNumber) -> all	Each row represents one specific driver's lap in one race. Position, lap time, sector times, tyre information, and track status describe that lap and are not treated as determinants.
PitStop	(raceId, driverId, lapNumber)	(raceId, driverId, lapNumber) -> durationMs, timeOfDay	A driver can make at most one PitStop during a particular lap, so the associated Lap key identifies the stop.
TelemetrySample	(raceId, driverId, lapNumber, offsetMs)	(raceId, driverId, lapNumber, offsetMs) -> all	Each row represents one telemetry reading at one instant within a specific lap. No proper subset of the key identifies a particular telemetry sample.
User	userId; username; email	userId -> all; username -> all; email -> all	username and email are unique across User records, so they are

			alternate candidate keys. All determinants are candidate keys.
Question	questionId	questionId -> all	sourceKey is unique when present for generated questions, but it is optional and is therefore not treated as a candidate key for every row. The optional Lap reference and authorId do not determine the other Question attributes.
AnswerOption	(questionId, label)	(questionId, label) -> text, isCorrect	A label such as A or B identifies an option only within its Question. The complete key is the determinant.
Quiz	quizId	quizId -> all	ownerId, driverId, constructorId, and title can be shared by multiple quizzes and do not determine the remaining attributes.
QuizQuestion	(quizId, questionId)	(quizId, questionId) -> position	Each row represents one Question included in one Quiz. Within a quiz, each included Question has one position, which is determined by the complete primary key.
QuizAttempt	attemptId	attemptId -> all	A User can attempt the same Quiz multiple times, so (userId, quizId) is not a key. attemptId is the determinant.
AttemptAnswer	(attemptId, questionId)	(attemptId, questionId) -> selectedLabel, timeTakenMs	Each row represents one Question answered during one QuizAttempt. Correctness is not stored because it is obtained through the selected AnswerOption.

Favorite	(userId, questionId)	(userId, questionId) -> savedAt	savedAt describes when one particular User saved one particular Question, so it depends on the complete key.
----------	----------------------	---------------------------------	--

Normalization Conclusion

All 21 relations in the final schema satisfy BCNF because every non-trivial functional dependency has a candidate key or superkey as its determinant.

The main decomposition required during normalization was separating country from Circuit into the Location relation because of the dependency location -> country.

Relational Schema

Multiple [PK] columns form one composite primary key.

Foreign key columns that reference composite keys act together. For example, (raceId, driverId) in Lap references RaceResult, while (raceId, driverId, lapNumber) in PitStop, TelemetrySample, and Question references Lap.

[UNIQUE] marks single-column unique attributes shown in the conceptual design. Optional relationships use nullable foreign keys. Question's Lap reference is either entirely NULL or entirely populated.

Formula 1 Data Relations

Driver(driverId:INT [PK], driverRef:VARCHAR(100) [UNIQUE], code:VARCHAR(3), permanentNumber:INT, forename:VARCHAR(100), surname:VARCHAR(100), dateOfBirth:DATE, nationality:VARCHAR(100), wikiUrl:VARCHAR(2048))

Season(year:INT [PK], wikiUrl:VARCHAR(2048))

Location(location:VARCHAR(100) [PK], country:VARCHAR(100))

Circuit(circuitId:INT [PK], circuitRef:VARCHAR(100) [UNIQUE], name:VARCHAR(255), location:VARCHAR(100) [FK to Location.location], latitude:DECIMAL(10,7), longitude:DECIMAL(10,7), altitudeM:INT, wikiUrl:VARCHAR(2048))

Constructor(constructorId:INT [PK], constructorRef:VARCHAR(100) [UNIQUE], name:VARCHAR(255), nationality:VARCHAR(100), wikiUrl:VARCHAR(2048))

Race(raceId:INT [PK], year:INT [FK to Season.year], circuitId:INT [FK to Circuit.circuitId], round:INT, name:VARCHAR(255), raceDate:DATE, raceTime:TIME, wikiUrl:VARCHAR(2048))

Qualifying(raceId:INT [PK] [FK to Race.raceId], driverId:INT [PK] [FK to Driver.driverId], position:INT, q1TimeMs:INT, q2TimeMs:INT, q3TimeMs:INT)

DriverStanding(year:INT [PK] [FK to Season.year], driverId:INT [PK] [FK to Driver.driverId], position:INT, points:DECIMAL(10,2))

ConstructorStanding(year:INT [PK] [FK to Season.year], constructorId:INT [PK] [FK to Constructor.constructorId], position:INT, points:DECIMAL(10,2))

RaceResult(raceId:INT [PK] [FK to Race.raceId], driverId:INT [PK] [FK to Driver.driverId], constructorId:INT [FK to Constructor.constructorId], carNumber:INT, gridPosition:INT, finishPosition:INT, positionOrder:INT, points:DECIMAL(10,2), lapsCompleted:INT, raceTimeMs:INT, status:VARCHAR(100))

Lap(raceId:INT [PK] [FK to RaceResult.raceId], driverId:INT [PK] [FK to RaceResult.driverId], lapNumber:INT [PK], position:INT, lapTimeMs:INT, sector1Ms:INT, sector2Ms:INT, sector3Ms:INT, tyreCompound:VARCHAR(30), tyreLifeLaps:INT, trackStatus:VARCHAR(20), speedTrapKph:DECIMAL(7,3))

PitStop(raceId:INT [PK] [FK to Lap.raceId], driverId:INT [PK] [FK to Lap.driverId], lapNumber:INT [PK] [FK to Lap.lapNumber], durationMs:INT, timeOfDay:TIME)

TelemetrySample(raceId:INT [PK] [FK to Lap.raceId], driverId:INT [PK] [FK to Lap.driverId], lapNumber:INT [PK] [FK to Lap.lapNumber], offsetMs:INT [PK], speedKph:DECIMAL(7,3), rpm:INT, gear:INT, throttlePct:DECIMAL(5,2), brakeOn:BOOLEAN, drsState:INT, posX:DECIMAL(12,3), posY:DECIMAL(12,3))

Application Relations

User(userId:INT [PK], username:VARCHAR(50) [UNIQUE], email:VARCHAR(254) [UNIQUE], passwordHash:VARCHAR(255), country:VARCHAR(100), isAdmin:BOOLEAN, createdAt:TIMESTAMP)

Question(questionId:INT [PK], authorId:INT [FK to User.userId], raceId:INT [FK to Lap.raceId], driverId:INT [FK to Lap.driverId], lapNumber:INT [FK to Lap.lapNumber], text:TEXT, format:VARCHAR(30), category:VARCHAR(100), difficulty:VARCHAR(20), explanation:TEXT, visibility:VARCHAR(20), sourceKey:VARCHAR(255) [UNIQUE], createdAt:TIMESTAMP)

Quiz(quizId:INT [PK], ownerId:INT [FK to User.userId], driverId:INT [FK to Driver.driverId], constructorId:INT [FK to Constructor.constructorId], title:VARCHAR(255), description:TEXT,

quizType:VARCHAR(30), visibility:VARCHAR(20), category:VARCHAR(100), eraStartYear:INT, eraEndYear:INT, createdAt:TIMESTAMP)

QuizQuestion(quizId:INT [PK] [FK to Quiz.quizId], questionId:INT [PK] [FK to Question.questionId], position:INT)

AnswerOption(questionId:INT [PK] [FK to Question.questionId], label:VARCHAR(10) [PK], text:TEXT, isCorrect:BOOLEAN)

QuizAttempt(attemptId:INT [PK], quizId:INT [FK to Quiz.quizId], userId:INT [FK to User.userId], startedAt:TIMESTAMP, completedAt:TIMESTAMP)

AttemptAnswer(attemptId:INT [PK] [FK to QuizAttempt.attemptId], questionId:INT [PK] [FK to Question.questionId] [FK to AnswerOption.questionId], selectedLabel:VARCHAR(10) [FK to AnswerOption.label], timeTakenMs:INT)

Favorite(userId:INT [PK] [FK to User.userId], questionId:INT [PK] [FK to Question.questionId], savedAt:TIMESTAMP)